

DOCUMENTAÇÃO TÉCNICA

Modelo de Testes Unitários e de Integração

Aplicado a um Sistema de Defesa Civil contra Enchentes e Deslizamentos com Dados Geograficamente Mapeados

Projeto Acadêmico — Curso de Desenvolvimento de Software Multiplataforma (DSM)

Autor: Gabriel Henrique Lazaro Almeida

Agosto de 2026

Resumo

Este documento apresenta um modelo de estratégia de testes de software — unitários e de integração — aplicado a um sistema acadêmico de defesa civil voltado ao monitoramento e alerta de riscos de enchentes e deslizamentos a partir de dados geograficamente mapeados. A metodologia baseia-se na Pirâmide de Testes de Cohn/Fowler, no padrão Arrange-Act-Assert, em princípios de teste orientado a risco (ISTQB) e em práticas inspiradas em Test-Driven Development para a camada de regras de negócio. São detalhadas a arquitetura de referência do sistema, os critérios de escolha de ferramentas (JUnit 5, Mockito, AssertJ, Spring Boot Test, Testcontainers, WireMock, JaCoCo), exemplos de código, a organização de pastas, metas de cobertura, um pipeline de integração contínua e um cronograma de dez semanas, de forma a compor um modelo viável e reproduzível para um projeto de faculdade.

Palavras-chave: Teste de Software; Teste Unitário; Teste de Integração; Pirâmide de Testes; Dados Geoespaciais; Defesa Civil; Spring Boot.

Sumário

1. Introdução e Contexto do Projeto
2. Objetivos deste Documento
3. Metodologia
4. Visão Geral da Arquitetura Proposta
5. Fundamentos: Teste Unitário vs. Teste de Integração
6. Pirâmide de Testes Aplicada ao Projeto
7. Estratégia de Teste Unitário
8. Estratégia de Teste de Integração
9. Testando com Dados Geograficamente Mapeados
10. Estrutura de Pastas e Organização dos Testes
11. Cobertura de Código e Metas de Qualidade
12. Pipeline de Integração Contínua (CI)
13. Cronograma Sugerido para o TCC/Projeto de Faculdade
14. Conclusão
15. Referências

1. Introdução e Contexto do Projeto

Este documento apresenta um modelo viável de estratégia de testes de software — unitários e de integração — para um projeto acadêmico cujo domínio é a Defesa Civil, especificamente o monitoramento e alerta de riscos de enchentes e deslizamentos de terra a partir de dados geograficamente mapeados (dados georreferenciados).

A escolha desse domínio agrega complexidade real ao projeto: além das regras de negócio tradicionais, o sistema lida com dados espaciais (coordenadas, polígonos de risco, altimetria, pluviometria), integrações externas (APIs meteorológicas, bases geográficas como IBGE e CPRM) e requisitos de confiabilidade elevados, já que falhas podem comprometer alertas que protegem vidas. Isso torna a estratégia de testes um elemento central do projeto, e não apenas um item acessório de qualidade.

1.1 Nome de referência do projeto

Para fins deste documento, o sistema é referenciado como SIGVIA — Sistema Integrado de Geomonitoramento e Alerta (o nome pode ser adaptado ao título real escolhido para o projeto).

2. Objetivos deste Documento

- Definir os conceitos de teste unitário e teste de integração aplicados ao contexto do projeto.
- Propor uma arquitetura de referência para o sistema de defesa civil, servindo de base para os exemplos de teste.
- Apresentar um modelo viável e reproduzível de estratégia de testes, com ferramentas, exemplos de código e organização de pastas.
- Detalhar como testar componentes que dependem de dados geográficos mapeados (coordenadas, polígonos, camadas GIS).
- Fornecer metas de cobertura, pipeline de CI e cronograma compatível com um projeto de faculdade (TCC, projeto integrador ou disciplina de Engenharia de Software).

3. Metodologia

Esta seção descreve a metodologia empregada na definição da estratégia de testes, de modo que o modelo proposto não seja apenas uma lista de boas práticas, mas o resultado de um processo justificável e replicável por outros projetos acadêmicos.

3.1 Abordagem geral

A metodologia adotada combina três frentes complementares: (i) revisão da literatura técnica de referência sobre estratégia de testes; (ii) definição de critérios objetivos de classificação dos testes por escopo e velocidade; e (iii) priorização por risco, alocando maior esforço de teste nos componentes cuja falha teria maior impacto — no caso deste projeto, o cálculo do risco e o disparo do alerta.

3.2 Base conceitual — Pirâmide de Testes

O modelo estrutura-se sobre a Pirâmide de Testes, formulada por Mike Cohn e posteriormente popularizada e detalhada por Martin Fowler. O princípio central é manter uma base larga de testes unitários — rápidos e baratos de manter — reduzindo progressivamente a quantidade de testes conforme

aumenta o escopo e o custo de execução (integração e, no topo, ponta a ponta). Essa distribuição orienta as metas de proporção definidas na Seção 6 deste documento.

3.3 Padrão de escrita dos testes — Arrange, Act, Assert (AAA)

Todos os testes do projeto seguem a estrutura AAA, também difundida por Fowler: Arrange (preparar o cenário e os dados de entrada), Act (executar a ação ou o método sob teste) e Assert (verificar se o resultado corresponde ao esperado). Esse padrão é aplicado de forma explícita nos exemplos de código apresentados neste documento, tornando os testes mais legíveis e fáceis de manter por diferentes integrantes da equipe.

3.4 Desenvolvimento orientado a testes no domínio (TDD)

Para a camada de domínio — em especial as regras de cálculo do índice de risco — recomenda-se uma abordagem inspirada em Test-Driven Development (TDD), na qual o teste unitário é escrito antes da implementação da regra de negócio. Essa prática força a especificação clara do comportamento esperado (por exemplo, os limites que definem os níveis de risco baixo, moderado, alto e crítico) antes da codificação, reduzindo ambiguidades típicas de regras de negócio baseadas em múltiplos parâmetros físicos.

3.5 Teste baseado em risco (Risk-Based Testing)

Seguindo os princípios de teste baseado em risco descritos pelo ISTQB (International Software Testing Qualifications Board), o esforço de teste não é distribuído de forma uniforme entre os módulos do sistema. Componentes com maior probabilidade de falha e maior impacto — cálculo de risco, persistência geoespacial e disparo de alerta — recebem cobertura de teste mais rigorosa (metas descritas na Seção 11), enquanto módulos de menor criticidade recebem cobertura proporcionalmente menor.

3.6 Critérios de seleção das ferramentas

As ferramentas indicadas ao longo deste documento (JUnit 5, Mockito, AssertJ, Spring Boot Test, Testcontainers, WireMock, JaCoCo) foram selecionadas segundo três critérios: serem gratuitas e de código aberto — compatíveis com o orçamento de um projeto acadêmico; possuírem documentação oficial ativa e ampla adoção pela comunidade Java/Spring; e permitirem testes reproduzíveis em ambiente local e em integração contínua, sem depender de serviços externos pagos.

3.7 Verificação e validação do modelo

A viabilidade do modelo é verificada por meio de três mecanismos: execução automatizada dos testes a cada contribuição via pipeline de CI (Seção 12); acompanhamento de métricas objetivas de cobertura de código por camada (Seção 11); e revisão do fluxo completo por meio de testes de ponta a ponta no topo da pirâmide, garantindo que a integração entre todos os serviços reproduza o comportamento esperado em produção.

4. Visão Geral da Arquitetura Proposta

O sistema é modelado como um conjunto de microsserviços (Java/Spring Boot), alinhado ao estilo arquitetural já adotado em outros projetos do autor, com um frontend web/mobile para visualização em mapa.

Serviço	Responsabilidade
geo-data-service	Ingestão e normalização de dados geográficos (GeoJSON/Shapefile): altimetria, declividade do terreno, hidrografia.
clima-service	Integração com APIs externas de meteorologia (pluviometria, previsão de chuva acumulada).
risco-service	Cálculo do índice de risco de enchente/deslizamento por área, cruzando dados de geo-data-service e clima-service.
alerta-service	Geração e disparo de alertas (push, SMS, e-mail) para a população e para a Defesa Civil quando o risco ultrapassa limites.
gateway	Ponto único de entrada, autenticação e roteamento entre os serviços.
dashboard (frontend)	Mapa interativo com camadas de risco, histórico de ocorrências e painel para agentes da Defesa Civil.

Essa divisão em serviços é o que orienta a separação entre o que deve ser validado por teste unitário (regras de cálculo, parsing, validações) e o que exige teste de integração (persistência, comunicação entre serviços, chamadas a APIs externas).

5. Fundamentos: Teste Unitário vs. Teste de Integração

4.1 Teste Unitário

Verifica uma unidade isolada de código (um método, uma classe, uma regra de negócio), sem dependências externas reais. Dependências (bancos, APIs, outros serviços) são substituídas por dublês de teste (mocks/stubs). É rápido, determinístico e deve compor a maior parte da suíte de testes.

4.2 Teste de Integração

Verifica a colaboração real entre dois ou mais componentes: aplicação e banco de dados, aplicação e API externa, ou comunicação entre microsserviços via HTTP/mensageria. É mais lento e mais próximo do comportamento em produção, por isso deve ser usado com foco nos pontos de maior risco de falha.

Critério	Teste Unitário	Teste de Integração
Escopo	Uma classe/método isolado	Múltiplos componentes reais
Velocidade	Muito rápido (ms)	Mais lento (segundos)
Dependências	Mockadas	Reais ou containerizadas
Quando falha	Aponta o bug com precisão	Aponta a área/fronteira do problema
Exemplo no projeto	Cálculo do índice de risco a partir da declividade e chuva acumulada	Persistir uma zona de risco no MongoDB geoespacial e recuperá-la por consulta espacial

6. Pirâmide de Testes Aplicada ao Projeto

A estratégia segue a pirâmide de testes clássica, adaptada às proporções recomendadas para um projeto acadêmico de porte médio:

- Base (≈70%) — Testes Unitários: regras de risco, validação de coordenadas, parsing de GeoJSON, cálculo de distância/área.
- Meio (≈20%) — Testes de Integração: persistência geoespacial, integração entre risco-service e clima-service, chamadas ao gateway.
- Topo (≈10%) — Testes E2E/Ponta a Ponta: fluxo completo — dado climático chega, risco é calculado, alerta é disparado e aparece no dashboard.

Essa proporção evita duas armadilhas comuns em projetos de faculdade: suítes lentas e frágeis (excesso de testes E2E) e falsa sensação de segurança (testes unitários demais, sem nunca validar a integração real entre serviços).

7. Estratégia de Teste Unitário

6.1 O que testar

- Cálculo do índice de risco (ex.: combinação de declividade do terreno + volume de chuva acumulada em 24h/72h).
- Classificação do nível de alerta (baixo, moderado, alto, crítico) a partir do índice calculado.
- Validação de coordenadas geográficas (latitude/longitude dentro de limites válidos, formato correto).
- Parsing e transformação de arquivos GeoJSON recebidos do geo-data-service.
- Regras de negócio de disparo de alerta (ex.: não duplicar alerta para a mesma área em menos de X minutos).

6.2 Ferramentas recomendadas

Ferramenta	Uso
JUnit 5	Framework base de testes em Java
Mockito	Criação de mocks/stubs para isolar dependências (repositórios, clientes HTTP)
AssertJ	Asserções fluentes e legíveis (assertThat(...))
JaCoCo	Medição de cobertura de código

6.3 Exemplo — cálculo de risco de deslizamento

```
@ExtendWith(MockitoExtension.class)
class RiscoDeslizamentoServiceTest {

    private final RiscoDeslizamentoService service = new RiscoDeslizamentoService();

    @Test
    void deveClassificarRiscoComoCriticoQuandoDeclividadeEChuvaForemAltas() {
        // Arrange
        DadosTerreno terreno = new DadosTerreno(declividadeGraus: 38.0);
        DadosPluviometricos chuva = new DadosPluviometricos(acumulado72h: 180.0);
```

```

// Act
NivelRisco resultado = service.calcularRisco(terreno, chuva);

// Assert
assertThat(resultado).isEqualTo(NivelRisco.CRITICO);
}

@Test
void deveClassificarRiscoComoBaixoQuandoDeclividadeForPequena() {
    DadosTerreno terreno = new DadosTerreno(declividadeGraus: 5.0);
    DadosPluviometricos chuva = new DadosPluviometricos(accumulado72h: 10.0);

    NivelRisco resultado = service.calcularRisco(terreno, chuva);

    assertThat(resultado).isEqualTo(NivelRisco.BAIXO);
}
}

```

8. Estratégia de Teste de Integração

7.1 O que testar

- Persistência de zonas de risco em banco com suporte geoespacial (ex.: MongoDB com índice 2dsphere).
- Consultas espaciais (ex.: buscar todas as zonas de risco que contêm um determinado ponto/coordenada).
- Integração real entre risco-service e clima-service via HTTP (contrato de API, tratamento de erro/timeout).
- Integração com API externa de meteorologia (simulada via mock de servidor HTTP, não a API real, para evitar flakiness).
- Fluxo do alerta-service: da geração do alerta até a fila/tópico de notificação.

7.2 Ferramentas recomendadas

Ferramenta	Uso
Spring Boot Test (@SpringBootTest)	Sobe o contexto real da aplicação para o teste
Testcontainers	Sobe instâncias reais e efêmeras de MongoDB/Redis em Docker durante o teste
WireMock	Simula respostas da API externa de meteorologia sem depender da rede real
RestAssured / WebClient	Testa os endpoints REST do serviço de ponta a ponta

7.3 Exemplo — persistência geoespacial com Testcontainers

```

@SpringBootTest
@Testcontainers
class ZonaRiscoRepositoryIT {

    @Container
    static MongoDBContainer mongo = new MongoDBContainer("mongo:7.0");
}

```

```

@DynamicPropertySource
static void mongoProps(DynamicPropertyRegistry registry) {
    registry.add("spring.data.mongodb.uri", mongo::getReplicaSetUrl);
}

@Autowired
private ZonaRiscoRepository repository;

@Test
void devePersistirEConsultarZonaDeRiscoPorPontoGeografico() {
    // Arrange
    Point pontoBairro = new GeoJsonPoint(-45.8869, -23.1791); // exemplo: São José dos
Campos
    ZonaRisco zona = new ZonaRisco("Zona Norte", poligonoRisco, NivelRisco.ALTO);
    repository.save(zona);

    // Act
    List<ZonaRisco> encontradas = repository.findByAreaContendoPonto(pontoBairro);

    // Assert
    assertThat(encontradas).hasSize(1);
    assertThat(encontradas.get(0).getNivelRisco()).isEqualTo(NivelRisco.ALTO);
}
}

```

7.4 Exemplo — integração com API externa de clima via WireMock

```

@SpringBootTest
@AutoConfigureWireMock(port = 0)
class ClimaServiceClientIT {

    @Autowired
    private ClimaServiceClient clienteClima;

    @Test
    void deveTratarTimeoutDaApiExternaSemDerrubarOFluxoDeAlerta() {
        stubFor(get(urlPathMatching("/previsao/*"))
            .willReturn(aResponse().withFixedDelay(6000)));

        DadosPluviometricos resultado = clienteClima.consultarComFallback("SJC");

        assertThat(resultado).isEqualTo(DadosPluviometricos.indisponivel());
    }
}

```

9. Testando com Dados Geograficamente Mapeados

Componentes que lidam com dados geoespaciais exigem cuidados específicos de teste, além dos princípios gerais de unitário/integração:

8.1 Fontes de dados para teste

- Usar recortes reais e pequenos de dados públicos (ex.: shapefiles/GeoJSON do IBGE, CPRM ou Defesa Civil municipal) como massa de teste, salvos em `src/test/resources`.
- Criar fixtures sintéticas para casos extremos (coordenadas nos limites do município, polígonos degenerados, dados ausentes).
- Evitar depender da API externa real durante os testes automatizados — usar cópias estáticas ou WireMock.

8.2 Pontos de atenção específicos

- Precisão numérica: comparações de coordenadas/áreas devem usar tolerância (ex.: `assertThat(valor).isCloseTo(esperado, within(0.0001))`), nunca igualdade exata de ponto flutuante.
- Sistemas de referência: garantir que todos os dados estejam no mesmo sistema de coordenadas (ex.: WGS84) antes de comparar; testar a conversão quando houver.
- Casos de borda geográfica: ponto exatamente sobre a fronteira de uma zona de risco, polígonos que se sobrepõem, coordenadas inválidas (fora de -90/90 e -180/180).
- Volume de dados: um teste de integração pode validar desempenho aceitável ao consultar um número realista de zonas de risco (ex.: todos os bairros de uma cidade).

8.3 Exemplo — validação de coordenada (teste unitário)

```
@Test
void deveRejeitarCoordenadaComLatitudeForaDoIntervaloValido() {
    assertThatThrownBy(() -> Coordenada.of(120.0, -45.88))
        .isInstanceOf(CoordenadaInvalidaException.class)
        .hasMessageContaining("latitude");
}

@Test
void deveCalcularAreaDoPoligonoComTolerancia() {
    double area = GeoUtils.calcularAreaKm2(poligonoZonaNorte);
    assertThat(area).isCloseTo(4.72, within(0.01));
}
```

10. Estrutura de Pastas e Organização dos Testes

```
risco-service/
├── src/
│   ├── main/java/com/sigvia/risco/
│   │   ├── domain/           (regras de negócio: cálculo de risco)
│   │   ├── geo/              (utilitários geoespaciais)
│   │   ├── client/           (clientes HTTP para clima-service)
│   │   ├── repository/       (persistência MongoDB)
│   │   └── controller/       (endpoints REST)
│   └── test/
│       ├── java/com/sigvia/risco/
│       │   ├── domain/       (testes unitários)
│       │   ├── geo/          (testes unitários)
│       │   ├── client/       (testes de integração, sufixo *IT)
│       │   └── repository/    (testes de integração, sufixo *IT)
│       ├── resources/
│       │   ├── geojson/      (massa de dados geográficos de teste)
│       │   └── application-test.yml
│       └── build.gradle
└── build.gradle
```

- Convenção de nomes: `*Test` para testes unitários (rodam sempre); `*IT` (Integration Test) para testes de integração (rodam em fase separada).
- Separar as fases no Gradle/Maven: ``test`` executa apenas `*Test`; ``integrationTest`` executa apenas `*IT` — evita que a suíte unitária fique lenta no dia a dia.

11. Cobertura de Código e Metas de Qualidade

Camada	Meta de cobertura sugerida	Justificativa
domain (regras de risco)	≥ 85%	Núcleo crítico do sistema, impacta diretamente a segurança das pessoas
geo (utilitários geoespaciais)	≥ 80%	Lógica matemática sensível a casos de borda
client/repository	≥ 60% (via testes de integração)	Cobertura vem principalmente dos testes *IT, não unitários
controller	≥ 60%	Focar nos contratos de entrada/saída da API

Recomenda-se configurar o JaCoCo para falhar o build caso a cobertura da camada domain caia abaixo da meta, reforçando a disciplina de testes ao longo do desenvolvimento do TCC.

12. Pipeline de Integração Contínua (CI)

Um pipeline simples no GitHub Actions garante que a estratégia de testes seja executada automaticamente a cada contribuição, o que é especialmente valioso em projetos de faculdade com múltiplos integrantes.

```
name: build-and-test
on: [push, pull_request]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: ./gradlew test jacocoTestReport

  integration-tests:
    runs-on: ubuntu-latest
    needs: unit-tests
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: ./gradlew integrationTest # sobe Testcontainers via Docker
```

- Job unit-tests: rápido, roda a cada commit/PR.
- Job integration-tests: depende do sucesso dos testes unitários, roda com Docker disponível (necessário para Testcontainers).
- Relatórios de cobertura (JaCoCo) podem ser publicados como artefato do workflow para acompanhamento pela equipe/orientador.

13. Cronograma Sugerido para o Projeto de Faculdade

Semana	Atividade
1–2	Definição da arquitetura, modelagem dos dados geográficos e das regras de risco
3–4	Implementação do domain e testes unitários das regras de cálculo de risco

Semana	Atividade
5–6	Implementação da persistência geoespacial e testes de integração (Testcontainers)
7	Integração risco-service ↔ clima-service, com WireMock nos testes
8	Implementação do alerta-service e testes do fluxo de disparo
9	Configuração do pipeline de CI e revisão da cobertura de testes
10	Testes E2E do fluxo completo e ajustes finais para a entrega/apresentação

14. Conclusão

O modelo apresentado combina fundamentos consolidados de teste de software (pirâmide de testes, isolamento por mocks, uso de containers efêmeros) com as particularidades de um domínio de defesa civil baseado em dados geográficos. Ele é viável para um projeto de faculdade porque usa ferramentas gratuitas e amplamente documentadas (JUnit, Mockito, Testcontainers, WireMock, JaCoCo, GitHub Actions), se apoia na arquitetura de microsserviços já familiar ao autor, e distribui o esforço de forma realista ao longo de um cronograma de 10 semanas.

Ao seguir esta estratégia, o projeto ganha não apenas qualidade técnica, mas também um argumento forte de defesa acadêmica: a confiabilidade dos alertas de risco — cujo impacto é a segurança de vidas — é sustentada por evidências concretas de teste em cada camada do sistema.

15. Referências

BECK, K. Test-Driven Development: By Example. Boston: Addison-Wesley, 2002.

COHN, M. Succeeding with Agile: Software Development Using Scrum. Boston: Addison-Wesley, 2009.

FOWLER, M. TestPyramid. martinowler.com, 2012. Disponível em: <https://martinfowler.com/bliki/TestPyramid.html>.

VOCKE, H. The Practical Test Pyramid. martinowler.com, 2018. Disponível em: <https://martinfowler.com/articles/practical-test-pyramid.html>.

ISTQB — International Software Testing Qualifications Board. Glossary of Testing Terms. Disponível em: <https://glossary.istqb.org/>.

JUNIT TEAM. JUnit 5 User Guide. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>.

MOCKITO. Mockito Framework Documentation. Disponível em: <https://site.mockito.org/>.

ASSERTJ. AssertJ — Fluent assertions for Java. Disponível em: <https://assertj.github.io/doc/>.

SPRING TEAM. Testing — Spring Boot Reference Documentation (Testcontainers integration). Disponível em: <https://docs.spring.io/spring-boot/reference/testing/testcontainers.html>.

TESTCONTAINERS. Testcontainers — Documentation. Disponível em: <https://testcontainers.com/>.

WIREFMCK. WireMock Documentation. Disponível em: <https://wiremock.org/docs/>.

JACOCO. JaCoCo Java Code Coverage Library. Disponível em: <https://www.jacoco.org/jacoco/>.

IBGE — Instituto Brasileiro de Geografia e Estatística. Bases cartográficas e geociências. Disponível em: <https://www.ibge.gov.br/geociencias/>.

CPRM — Serviço Geológico do Brasil. Cartas e dados de suscetibilidade a movimentos de massa e inundação. Disponível em: <https://www.cprm.gov.br/>.